

Rethinking the Render-Display Pipeline to Reduce Latency in Interaction

Evan Jonson
North Carolina State University
Raleigh, USA
ecjonson@ncsu.edu

Aaron Fulmer
Francis Marion University
Florence, USA
North Carolina State University
Raleigh, USA
aaron.fulmer@fmarion.edu

Hee-Jin Choi
Sejong University
Seoul, Republic of Korea
hjchoi@sejong.ac.kr

Hyoungsik Nam
Kyung Hee University
Seoul, Republic of Korea
hyoungsiknam@khu.ac.kr

Benjamin Watson
North Carolina State University
Raleigh, USA
bwatson@ncsu.edu

CCS Concepts

• Applied computing → Computer games; • Hardware → Emerging interfaces; • Human-centered computing → Displays and imagers.

Keywords

latency, frameless, renderer, display, integration, interaction, alternative scan, display scan, games, esports

ACM Reference Format:

Evan Jonson, Aaron Fulmer, Hee-Jin Choi, Hyoungsik Nam, and Benjamin Watson. 2026. Rethinking the Render-Display Pipeline to Reduce Latency in Interaction. In *Special Interest Group on Computer Graphics and Interactive Techniques Conference Emerging Technologies (SIGGRAPH Emerging Technologies '26)*, July 19–23, 2026, Los Angeles, CA, USA. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3799826.3812437>

1 Abstract

Over fifty years ago, rendering and display systems were designed for latency-free, recorded media. Yet today's interactive systems — which suffer significantly from latency — are still based on the old latency-free design assumptions. We propose an alternative: close integration of render and display that reduces interactive latency far below delays in current systems. By combining just-in-time input sampling; frameless, image-order rendering; and random-access displays supporting flexible scanning; we eliminate frame and synchronization delays, mitigate refresh delays, and reduce click-to-photon latency below 2 ms. We will demonstrate a low-resolution hardware prototype of this integrated pipeline supporting the game Tetris, along with a high-resolution interactive simulator of alternative display scans supporting more modern games. Together, these demonstrations will offer a compelling glimpse of a low-latency, much more interactive future for computer graphics.

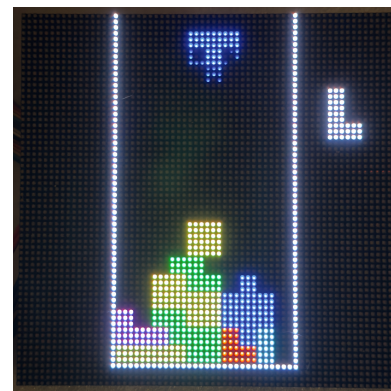


Figure 1: A moment in motion from the game Tetris, as depicted by our prototype, frameless architecture. This image was captured with shutter speed 1/24s, ISO 1000.



Figure 2: Example screen tearing frame using top-to-bottom raster scan (left) and stochastic scan (right) as shown in Borderlands 2 [Gearbox Software 2012]

2 Introduction

Modern displays were designed for latency-free recorded media, with modularity between renderer and display in mind. Yet today, most displays are used in integrated devices (smart phones, laptops, gaming handhelds) and in an interactive setting, in which a strong



This work is licensed under a Creative Commons Attribution 4.0 International License. *SIGGRAPH Emerging Technologies '26, Los Angeles, CA, USA*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2549-4/26/07
<https://doi.org/10.1145/3799826.3812437>

Table 1: NVIDIA LDAT[NVIDIA 2020] Click-to-Photon Latency Measurements of the integrated renderer and display

scan	standard: framed, synchronized, 60Hz						prototype: frameless, integrated					
	progressive			stochastic			progressive			stochastic		
LDAT placement	top left	center	bottom right	top left	center	bottom right	top left	center	bottom right	top left	center	bottom right
avg ms	26.8	32.7	41.6	24.5	26.5	29.9	8.1	8.2	9.1	1.8	1.6	2.0
stdev	4.8	5.0	4.9	4.3	4.9	4.7	4.9	5.2	5.2	0.8	0.8	1.1
min ms	18.1	24.3	33.1	17.6	17.6	21.8	0.7	0.8	0.8	0.7	0.6	0.7
max ms	35.0	40.9	49.4	33.6	34.5	38.1	17.5	17.4	17.4	4.7	3.8	4.8
samples	100	100	100	100	100	100	100	100	100	100	100	100

indicator of usability is interactive (or *click-to-photon*) latency – the delay between user input and photon emission on the display. Gaming displays have reduced interactive latency by increasing refresh rates, yet this display latency is only one piece of the delay users experience. It is time to rethink the render-display pipeline in integrated devices and prioritize user experience in interactive applications.

Current display systems have baked in the assumption that we need frames, which relies on buffering and the synchronization between display and renderer. The system maintains two buffers in memory where images are rendered into the *render* (or *back*) *buffer*, while the display scans from a second buffer, known as the *display* (or *front*) *buffer*. Once a frame render is complete, the two buffers are swapped. This technique is called *double-buffering*. By the time a frame reaches the display, it is already one frame time old (frame times vary depending on content complexity, but 60Hz frame rates are common, introducing 16.67ms of frame time latency) because the information was stored in a buffer until all of it was collected before any of it could be used to update the display. Even this simple buffering delay can be noticeable, with research suggesting that even as little delay as 30 ms can harm interaction, and even 10 ms can be perceivable [Ng and Dietz 2014] [Spjut et al. 2019].

This paper proposes two modern approaches to reduce interactive latency and presents a low-resolution hardware prototype and a high-resolution software simulator to provide E-tech attendees with the opportunity to experience these new approaches with a hands-on comparison of the interactive latency gains and the visual differences they create.

3 A Frameless Render-Display Architecture

To address interactive latency, we propose a tighter integration of renderer and display [Jonson et al. 2026]. This architecture discards the idea of a double- or even single-buffered system, eliminating latency introduced by frames and their buffers entirely. To achieve this, we take advantage of a property of image-order renderers, such as ray tracers, that until now has largely been overlooked: pixels are ready for display immediately upon computation, and do not need to be buffered as a frame.

First, sample input and application state, then render a single pixel, then send the computed color immediately to the display before proceeding. This method fully realizes a frameless renderer that can send individual pixels to the display, and a display that can light individual pixels as they are received, without either side waiting for an entire frame. Note that this display is now frameless – it does not utilize a frame buffer – every succeeding pixel represents

a slightly different and more up-to-date moment, with input and state sampled just in time.

An ideal architecture would render and light pixels in arbitrary order determined by the GPU, ensuring that pixels represent game state with the most accuracy possible. In this paper, we follow a fixed per-pixel random (stochastic) scan pattern, determined at initialization. We recognize that such a substantial change to the render-display pipeline would be very difficult, so we also propose a milder approach: one in which only the order with which pixels are updated on a display is modified, as explained in the next section.

4 An Alternative Stochastic Scan

We suggest that an alternative stochastic display scan pattern can avoid synchronization delay, distribute information with lower latency throughout the image, and smooth motion without distracting tears [Fulmer et al. 2025].

In the stochastic ordering, the next pixel updated is chosen randomly, with each pixel equally likely to be visited next in the order. No pixel is visited again until all pixels in the frame have already been updated. This stochastic order is pre-computed and well-known to both computer and display. Figure 2 (right) shows this stochastic scan when synchronization is turned off.

When the computer and display are unsynchronized to avoid introducing new delays, a stochastic ordering has a number of benefits. To begin, since multipurpose displays cannot know which display regions are most important, low-latency updates are distributed equally across the display. Thus whether gaming, conferencing, or writing, users will find low-latency information to assist them.

In addition, temporal artifacts are less distracting. Rather than temporal aliasing manifesting as a distinct region of pixels disjoint from those above them, a stochastic ordering would have this temporal aliasing appear as something closer to visual noise, which our visual system is far more tolerable of [Cook 1986].

5 Interactive Prototype

Unfortunately, modern LCD and OLED display driving components cannot support either a stochastic scan or an integrated render-display pipeline. Thus, we have created prototypes to evaluate how the ideas described in Sections 4 and 3 may impact both the cohesion of the resulting images and their latency.

5.1 A Frameless and Integrated Render-Display Prototype

We implement our integrated render-display prototype using a Raspberry Pi Pico 2 RP2350 and several small LED panels tiled together – 4 Waveshare HUB75 RGB LED matrix panels, each with 64x64 pixels, 2mm pitch and a 1/32 scan rate, for a total resolution of 128x128. The Pico 2 features two ARM Cortex-M33 cores that can operate at clock speeds well above 150MHz, as well as a parallel, deterministic programmable I/O subsystem, perfect for driving hardware at a low level. We use the Pico to run our application, render it, and drive the display. The Pico clock is fast enough to sustain scanout at rates sufficient to simulate a fully realized stochastic scan, or any other scan pattern.

We chose the HUB75 hardware and protocol because it is line-addressable and can operate at extremely high refresh rates. This enables us to access and light individual pixels as they are computed following a stochastic scan pattern. The Waveshare display is multiplexed, meaning it addresses and lights multiple lines simultaneously – not the entire panel. To simulate a single-pixel update, we write a single newly rendered pixel into a small working buffer containing the current image, then immediately scan out the entire line that contains it, with the paired line blacked out.

Figure 1 shows a moment in motion from the game Tetris, as depicted by our prototype. Note that the frameless architecture results in less well defined edges, since each pixel represents a slightly different moment in time. Here you can see where the tetris piece exists in the game state, and which pixels most accurately represent that state. We postulate that this effect is desired by users who value interactive latency over synchronization techniques, such as VSYNC. To quantify this, we also include Table 1, which details latency measured through our prototype when using a framed system that scans pixels to the display in the typical raster order as compared to when we use a stochastic scan ordering, a frameless system, or both. We used NVIDIA's Latency and Display Analysis Tool (LDAT) [NVIDIA 2020] placed at different regions on the display to obtain latency measurements for each region.

5.2 An Alternative Scanning Simulator

We designed our software display simulator with the intention that we could continue to design and implement new display scan orderings with relative ease. We use this same simulator to simulate a number of orderings, but we will focus on the stochastic and raster scan patterns for the purposes of this paper.

To begin, the GPU renders a frame and sends it over HDMI in normal raster order N . Next, custom software intercepts the frame using a capture card, reorders its pixels into simulated (e.g. stochastic) order S , and places the result into a memory buffer. A separate software process, which runs at a different rate, then accesses the buffer, reorders the pixels from order S back to original order N , and then sends the resulting frame to the physical display, where users can see the results.

With the simulator's two processes running at different, unsynchronized speeds, we can simulate the interaction of synchronized or unsynchronized display with stochastic and other scanning orders, including their effects on both visual quality and delay. Note that our prototype will have higher delays than a fully realized

hardware implementation, in which several of the software steps would not be necessary. Using our simulator connected to a ROG PG258 display – a 240Hz gaming monitor – we can accurately simulate the temporal aliasing (e.g. screen tearing) that occurs during unsynchronized display with alternative scans. We are confident that the prototype is a valuable proof-of-concept as the images that appear on the physical display accurately represent the visual effects of alternative scanning orders with unsynchronized display.

6 Future Hardware Realization

To achieve the true potential of a frameless and integrated render-display architecture, the renderer and display must be more tightly integrated, and both hardware and software must change. Renderers must be able to provide pixels immediately upon computation without waiting for frame completion, and the display must be able to address and light pixels individually and in arbitrary order. This sort of random access to display pixels is particularly difficult to achieve in hardware.

To implement stochastic scanning in hardware, GPUs would likely not need to change; drivers would simply apply the stochastic pattern to a frame before it gets serialized and sent to the display. Displays, however, would need more significant change. Random pixel access would be ideal, but difficult. A more modest solution would rewire display row buffers to connect pixels at random vertical locations, but even this could prove difficult. Yet should this be achieved, all other aspects of the display should remain the same, so stochastic scanning displays should run at the same speeds as current raster scanning displays. Emerging micro LED displays may be particularly suitable to stochastic scanning.

7 Experience

Our goal at E-Tech is to provide attendees with a hands-on experience of a display systems designed for ultra-low interactive latency. Attendees will be provided the opportunity to interact with each of the options we presented in Section 5.

Attendees who experience the high-resolution simulator will be presented with a choice of games to play with three different scans – raster scan with VSYNC, raster scan without VSYNC, and stochastic scan without VSYNC. All three will be implemented with our prototype to ensure that delays are comparable across all of the scan types. Although our simulator does not reflect the latency gains of a fully realized version of this architecture, players will get to experience the full effect of the visual quality of this architecture at a high-resolution.

Attendees who select the low-resolution prototype will be presented with a choice of playing either Tetris or Pong on our prototype LED panel with an Xbox controller. Players will be provided with a choice of up to four different architectures – standard framed and buffered with VSYNC (raster or stochastic) or our frameless, integrated system (raster or stochastic) which decreases interactive latency. Although our prototype is low-resolution and currently only supports simple games, players will get to experience the full effect of near microsecond interactive latency.

References

- Robert Cook. 1986. Stochastic Sampling in Computer Graphics. In *ACM Transactions on Graphics*, Vol. 5. 51–72.

- Aaron Fulmer, Hee-Jin Choi, and Benjamin Watson. 2025. Stochastic Scanning for Unsynchronized, Low-Latency Display Without Tearing. In *SID Symposium Digest of Technical Papers*, Vol. 56. Wiley Online Library, 1885–1888.
- Gearbox Software. 2012. *Borderlands 2*. Video game.
- Evan Jonson, Aaron Fulmer, Hyoungsik Nam, Hee-jin Choi, and Benjamin Watson. 2026. Ultra-Low Latency with Frameless Render-Display Integration. In *SID Symposium Digest of Technical Papers*. Wiley Online Library.
- Albert Ng and Paul H Dietz. 2014. The effects of latency and motion blur on touch screen user experience. *Journal of the Society for Information Display* 22, 9 (2014), 449–456.
- NVIDIA. 2020. NVIDIA Latency and Display Analysis Tool. <https://developer.nvidia.com/nvidia-latency-display-analysis-tool>
- Josef Spjut, Ben Boudaoud, Kamran Binaee, Jonghyun Kim, Alexander Majercik, Morgan McGuire, David Luebke, and Joohwan Kim. 2019. Latency of 30 ms Benefits First Person Targeting Tasks More Than Refresh Rate Above 60 Hz. In *SIGGRAPH Asia 2019 Technical Briefs* (Brisbane, QLD, Australia) (SA '19). Association for Computing Machinery, New York, NY, USA, 110–113. doi:10.1145/3355088.3365170